CUDA on Google Colab

Your first notebook, step by step: create a notebook, get a free NVIDIA GPU, and compile and run your first CUDA program. No GPU on your own computer is needed, only a Google account and a browser.

1. What Colab gives you
2. Create your notebook
3. Ask for a GPU
4. Check that the GPU is there
5. Write your first CUDA program
6. Compile and run it
7. Read the output
8. Experiments to try
9. When something goes wrong
10. Next steps

1 What Colab gives you

Google Colab is a free service that runs *notebooks* in your browser on a machine in Google's cloud. A notebook is a list of **cells**: text cells (explanations) and code cells (that you run one at a time with the ▶ button or **Shift + Enter**).

Colab notebooks are Python notebooks, but they can also run commands on the Linux machine behind them. That is all we need for CUDA:

In a code cell	What it does
`%%writefile hello.cu` (first line of the cell)	Saves the rest of the cell as the file `hello.cu` on the Colab machine. It does not run anything.
`!nvcc hello.cu -o hello`	The `!` sends the line to the machine's shell. `nvcc` is NVIDIA's CUDA compiler; it turns `hello.cu` into a program called `hello`.
`!./hello`	Runs the program. Its output appears below the cell.

Why Colab?

CUDA programs need an NVIDIA GPU. Most laptops (and every Mac) do not have one. Colab lends you one for free, with the NVIDIA driver and the CUDA toolkit already installed. If your own computer has an NVIDIA GPU, you can also install the CUDA Toolkit and use the same `nvcc` commands in a terminal.

2 Create your notebook

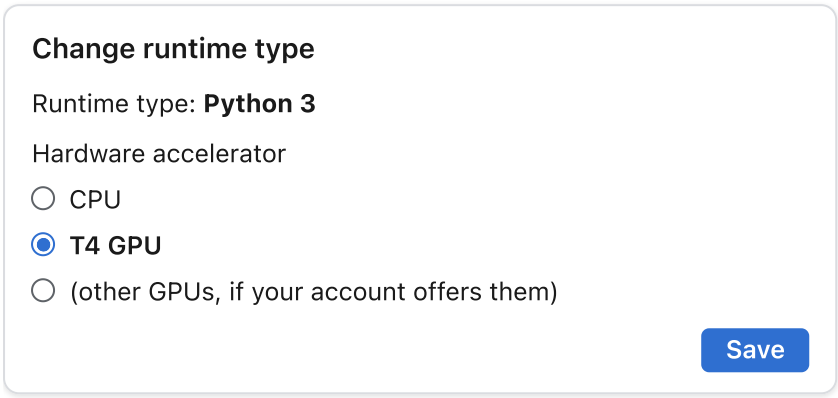
1. Go to **colab.research.google.com** and sign in with your Google account.
2. Click **New notebook** (or **File** → **New notebook in Drive**).
3. Click the title at the top left (`Untitled0.ipynb`) and rename it, for example `cuda_hello.ipynb`.

The notebook is saved automatically in your Google Drive, in the folder **Colab Notebooks**.

3 Ask for a GPU

A new notebook runs on a machine **without a GPU**. "Enabling the GPU" is not something your code does: you ask Colab for a different machine, one that has an NVIDIA GPU.

`Runtime` → `Change runtime type`, choose **T4 GPU** and click **Save**:



Do this first.
Changing the runtime restarts the machine and deletes the files written on it. If you change it later, run the notebook again from the top.

4 Check that the GPU is there

Click `+ Code` to add a code cell, type these two lines and run it (▶):

code cell

```
!nvidia-smi
!nvcc --version
```

`nvidia-smi` lists the GPU of the machine and `nvcc --version` prints the version of the CUDA compiler. You should see something like this (numbers will differ):

```
| NVIDIA-SMI 550.54.15    Driver Version: 550.54.15    CUDA Version: 12.4 |
|   0  Tesla T4              Off |  00000000:00:04.0 Off |              0 |
| N/A   44C    P8    9W /  70W |      0MiB / 15360MiB |      0%   Default |
...
Cuda compilation tools, release 12.5, V12.5.82
```

Ready when you see "Tesla T4".
If you get `nvidia-smi: command not found` or *No devices were found*, the runtime has no GPU: go back to step 3.

5 Write your first CUDA program

Add a new code cell, paste the program below and run the cell. The first line, `%%writefile hello.cu`, must be the very first line of the cell; it saves the code as `hello.cu` and prints *Writing hello.cu*.

code cell

```
%%writefile hello.cu
#include <cstdio>

__global__ void helloFromTheGpu() {
    int block = blockIdx.x;
    int thread = threadIdx.x;
    int globalThread = blockIdx.x * blockDim.x + threadIdx.x;
    printf("Hello from block %d, thread %d (global thread %d)\n",
           block, thread, globalThread);
}

int main() {
    printf("Hello from the CPU\n");

    const int blocks = 2;
    const int threadsPerBlock = 4;
    helloFromTheGpu<<<blocks, threadsPerBlock>>>();

    cudaError_t launchError = cudaGetLastError();
    if (launchError != cudaSuccess) {
        printf("The kernel could not start: %s\n", cudaGetErrorString(launchError));
        return 1;
    }

    cudaError_t runError = cudaDeviceSynchronize();
    if (runError != cudaSuccess) {
        printf("The kernel failed: %s\n", cudaGetErrorString(runError));
        return 1;
    }

    printf("The GPU finished: %d blocks x %d threads = %d threads said hello\n",
           blocks, threadsPerBlock, blocks * threadsPerBlock);
    return 0;
}
```

What the code means

Code	Meaning
`__global__ void helloFromTheGpu()`	A **kernel**: a function that runs on the GPU and is started from the CPU. Every GPU thread runs the whole function.
`blockIdx.x`, `threadIdx.x`, `blockDim.x`	Built-in variables. Each thread sees its own block number, its own number inside the block, and the number of threads per block. They are how a thread knows *which* thread it is.
`blockIdx.x * blockDim.x + threadIdx.x`	The thread's **global index** (0 to 7 here). Later this index chooses which array element each thread works on.
`printf` inside the kernel	Works on the GPU too: each thread prints its own line.
`helloFromTheGpu<<<2, 4>>>()`	The **launch**: start the kernel with a grid of 2 blocks of 4 threads each, 8 threads in total.

`cudaGetLastError()`	A launch returns nothing. This call says whether it could start (for example, too many threads per block).
`cudaDeviceSynchronize()`	The CPU does not wait for the GPU by itself. This makes it wait until the kernel has finished, so the GPU's `printf` lines appear before the program ends, and it reports errors that happened while the kernel ran.

6 Compile and run it

Add one more code cell and run it:

code cell

```
!nvcc -arch=native hello.cu -o hello && ./hello
```

- `nvcc hello.cu -o hello` compiles the file into the program `hello`.
- `-arch=native` compiles for the GPU that is in this machine (the T4). If your `nvcc` is old and does not know `native`, use `-arch=sm_75`, the T4's architecture.
- `&& ./hello` runs the program only if compiling worked.

7 Read the output

```
Hello from the CPU
Hello from block 1, thread 0 (global thread 4)
Hello from block 1, thread 1 (global thread 5)
Hello from block 1, thread 2 (global thread 6)
Hello from block 1, thread 3 (global thread 7)
Hello from block 0, thread 0 (global thread 0)
Hello from block 0, thread 1 (global thread 1)
Hello from block 0, thread 2 (global thread 2)
Hello from block 0, thread 3 (global thread 3)
The GPU finished: 2 blocks x 4 threads = 8 threads said hello
```

- The CPU line comes first, then **8 lines from 8 GPU threads**, then the CPU line after `cudaDeviceSynchronize()`.
- **The order of the GPU lines is not fixed.** Threads run in parallel and the GPU decides which block goes first; block 1 may print before block 0, and the order can change from one run to the next. Inside a block, threads are grouped in *warps* of 32, so here the 4 threads of a block usually print together.
- Thread numbers start at 0 in **every** block; the global index (0 to 7) is unique across the grid.

Congratulations: you ran code on a GPU.

Save your notebook (File → Save, or Ctrl/Cmd + S). It stays in Google Drive, in **Colab Notebooks**.

8 Experiments to try

Change the program in the `%%writefile` cell, run that cell again (to save the new version), then run the compile cell again.

Change	What you should see
`blocks = 3`, `threadsPerBlock = 5`	15 lines, global threads 0 to 14, thread numbers 0 to 4 in every block.
`blocks = 1`, `threadsPerBlock = 64`	64 lines. Look at the order: lines come in groups of 32 (one warp at a time).
`threadsPerBlock = 2048`	No GPU lines, and *The kernel could not start: invalid configuration argument*. A block can have at most 1024 threads. Without the `cudaGetLastError()` check, nothing would warn you.
Delete the line `cudaDeviceSynchronize()` (and its `if`)	Often the GPU lines disappear: the program ends before the GPU has finished printing. This is why the CPU must wait for the GPU.
In the kernel, print only if `globalThread % 2 == 0`	Only the even global threads print: every thread runs the same code but can take a different path depending on its index.

9 When something goes wrong

Symptom	Cause and fix
`nvidia-smi: command not found`, *No devices were found*, or *no CUDA-capable device is detected*	The runtime has no GPU. **Runtime** → **Change runtime type** → **T4 GPU**, then run everything again from the top.
hello.cu: No such file or directory	The `%%writefile` cell was not run (or the machine restarted and lost the file). Run it again, then compile.
UsageError: Line magic function `%%writefile` not found	`%%writefile` must be the **first** line of the cell, with nothing (not even a blank line) above it.
Only the CPU lines appear	The kernel did not run or the CPU did not wait. Check that `cudaDeviceSynchronize()` is there and look for an error message.
no kernel image is available for execution on the device	The code was compiled for a different GPU. Use `-arch=native` (or `-arch=sm_75` for a T4).
nvcc fatal: Unsupported gpu architecture 'compute_native'	That `nvcc` is too old for `-arch=native`: use `-arch=sm_75`.
Compile errors about `<<<` or `__global__`	The file must end in `.cu` and be compiled with `nvcc`, not `g++`.
Cannot connect to GPU backend or *usage limits*	Free Colab GPUs are limited. Wait a while, or close other Colab tabs (**Runtime** → **Disconnect and delete runtime**) and try again.

The notebook disconnected after a break	Idle sessions are stopped and their files deleted. Run the notebook again from the top (**Runtime** → **Run all**).

10 Next steps

- The course's runnable examples are in the repository folder `ec2/`, in numbered folders that go from the simplest kernel to the parallel patterns. Each folder has ready-made notebooks: start with `ec2/01_first_kernels/colab_1_step_by_step.ipynb`.
- To open one in Colab: **File** → **Upload notebook**, pick the `.ipynb` file, then set the runtime to **T4 GPU** (step 3) and run the cells in order.
- The course programs include a small header, `cuda_compat.h`, and launch kernels with `launch(kernel, grid, block, ...)` instead of `kernel<<<grid, block>>>(...)`: on Colab it is exactly the same launch, and it lets the same file also run on a laptop without an NVIDIA GPU. The notebooks explain it in their first cells.

CUDA on Google Colab: initial setup guide for the GPU part of the course. Colab's menus and free GPU limits can change over time; the steps above were written for the Colab interface of 2026.